

5. Kernel methods: support vector machines and reproducing kernel Hilbert spaces

14.10.2025

ETH zürich

D MATH

- ① Motivation: What are support vector machines?
- ② Relation to reproducing kernel Hilbert spaces
- ③ Learning with reproducing kernel Hilbert spaces:
 - Kernel functions, Mercer's Theorem and the Representer Theorem
 - Numerical aspects
 - Learning rates

Expected time: $\sim 90+45$ minutes

What are Support Vector Machines?

Quick answer:

- A method for (supervised) learning which started to raise attention in the early 1990s
- The name Support Vector Machine (SVM) arises from
 - *Machines* that turn inputs into outputs (as in *machine learning*)
 - In motivating classification tasks, certain *vectors* which "*support* the decision boundaries" occur.
- Compared to other frequently used methods, particular characteristics of kernel methods are:
 - In contrast to methods like neural networks, numerical optimization is tractable/well-understood for SVMs. Reliability and interpretability are thus better for SVMs.
 - The theory behind SVMs/kernel methods is frequently used throughout the field of machine learning (e.g., neural networks are often studied through the lens of kernel methods, as with the neural tangent kernel).

Motivating example

- Classification task
- We want to classify points in $\mathcal{X} \subseteq \mathbb{R}^2$ into one of two categories $\mathcal{Y} = \{-1, 1\}$

If you want to play with the python code used to generate the following slides, you can take a look at <https://jakevdp.github.io/PythonDataScienceHandbook/05.07-support-vector-machines.html>

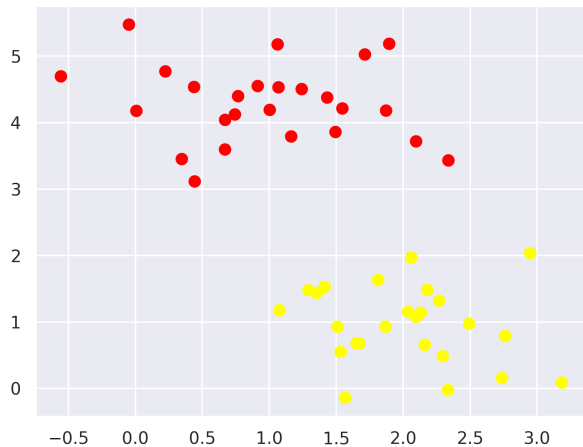


Figure: Points

$x_1, x_2, \dots, x_m \in \mathbb{R}^2$ with
corresponding labels

$y_1, y_2, \dots, y_m \in \{-1, 1\} =$
 $\{y, r\}$ (yellow or red).

Goal: Find a linear classifier to
separate yellow and red points.

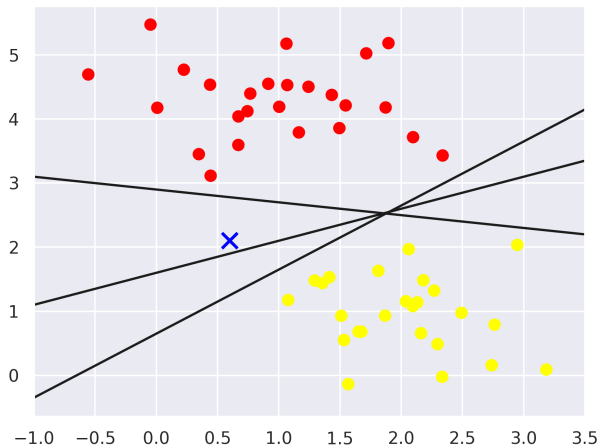


Figure: Different possible classifiers given by black lines. For a new point (e.g., blue cross), these may behave differently.
Question: Which linear separation to choose?

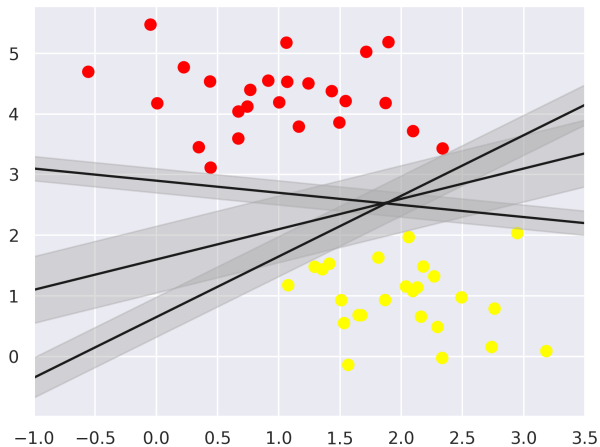


Figure: Geometric idea behind support vector machines: Each classifier has a corresponding "safety margin" (gray shaded area). We choose the classifier which **maximizes this margin**.

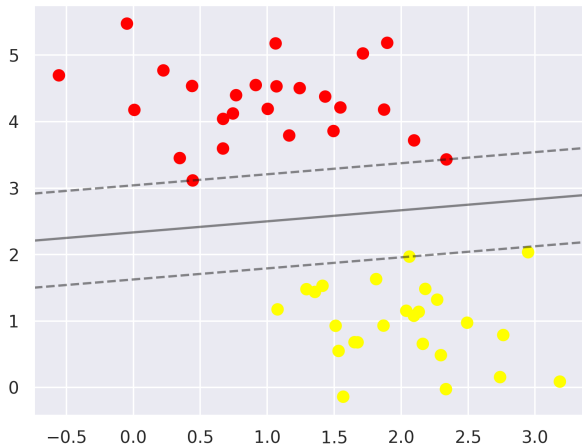


Figure: The optimal linear classifier for our data. The dashed lines are parallel shifts of the classifier line which touch both point clouds. These points are called the **support vectors** of this classifier.

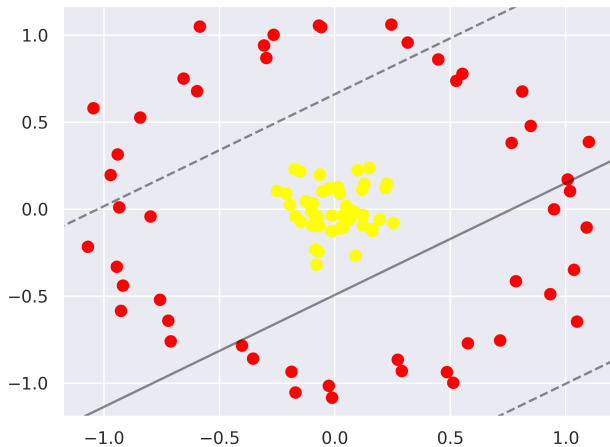


Figure: For data which is not linearly separable, we need more general ways to separate the data.

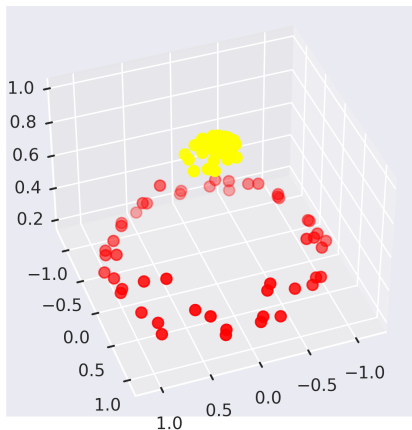


Figure: The idea behind support vector machines for nonlinear data is to **lift the data into a different space where it is linearly separable**.

For instance in this problem, we can lift $x_i \in \mathbb{R}^2$ to

$$\tilde{x}_i = (x_i, \exp(-\|x_i\|_2^2)) \in \mathbb{R}^3.$$

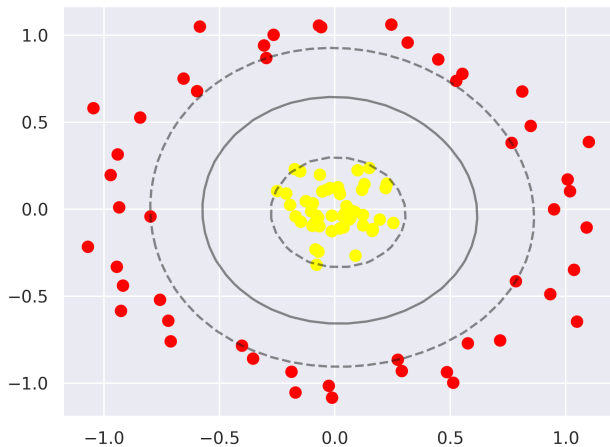


Figure: The linear classifier in the high-dimensional space corresponds to a nonlinear classifier in the low-dimensional space.

From geometric intuition to RKHS learning

- Introduction follows Chapter 1.3 of Steinwart and Christmann, 2008
- Let $(x_i, y_i) = (x_{i1}, \dots, x_{id}, y_i) \in \mathbb{R}^d \times \{-1, 1\}$, $i = 1, \dots, m$
- Assume **linear classification is possible**: i.e., assume there exist $w \in \mathbb{R}^d, b \in \mathbb{R}$ such that

$$\text{sign}(\langle w, x_i \rangle + b) = y_i \quad \text{for all } i = 1, \dots, m. \quad (1)$$

- Each classifier (w, b) has an affine decision hyperplane given by

$$H(w, b) := \{x \in \mathbb{R}^d : \langle w, x \rangle + b = 0\}.$$

- Choose a correct classifier $(w_D, b_D) \in \mathbb{R}^d \times \mathbb{R}$, normed to $\|w_D\|_2 = 1$, which maximizes the distance between data and decision hyperplane:

$$(w_D, b_D) = \arg \max_{\substack{(w, b) \in \mathbb{R}^d \times \mathbb{R}: \|w\|_2 = 1 \text{ and} \\ \text{sign}(\langle w, x_i \rangle + b) = y_i, i = 1, \dots, m}} \underbrace{\min_{i \in \{1, \dots, m\}, p \in H(w, b)} \|x_i - p\|_2}_{\text{distance between } \{x_1, \dots, x_m\} \text{ and } H(w, b)}$$

- **Equivalently** (PS3.1): Set $(w_D, b_D) = (w_D^* / \|w_D^*\|, b_D^* / \|w_D^*\|)$, where (w_D^*, b_D^*) are solutions to

$$\begin{aligned} & \min_{(w, b) \in \mathbb{R}^d \times \mathbb{R}} \langle w, w \rangle \\ & \text{subject to: } y_i(\langle w, x_i \rangle + b) \geq 1, \quad i = 1, \dots, m \end{aligned}$$

From geometric intuition to RKHS learning

$$\begin{aligned} \min_{(w,b) \in \mathbb{R}^d \times \mathbb{R}} \quad & \langle w, w \rangle \\ \text{subject to:} \quad & y_i(\langle w, x_i \rangle + b) \geq 1 \quad i = 1, \dots, m \end{aligned}$$

- In general situations, there are two issues
 - 1 linear separation may be impossible (we cannot satisfy the constraint)
 - 2 we may not want to satisfy the constraint (overfitting)
- Solutions using RKHS learning:
 - 1 We lift x_i into a (possibly infinite-dimensional) Hilbert space H_0 via a *feature map* $\Phi : \mathbb{R}^d \rightarrow H_0$.
As a classifier we use $w \in H_0$ instead of $w \in \mathbb{R}^d$.
 - 2 Introduce slack variables $\xi_i \geq 0$ which allow us to violate the constraint in exchange for a penalty in the objective value
- These solutions lead to the adjusted optimization problem

$$\begin{aligned} \min_{(w,b,\xi) \in H_0 \times \mathbb{R} \times \mathbb{R}_+^m} \quad & \langle w, w \rangle + C \sum_{i=1}^m \xi_i \\ \text{subject to:} \quad & y_i(\langle w, \Phi(x_i) \rangle + b) \geq 1 - \xi_i \quad i = 1, \dots, m \end{aligned} \tag{2}$$

From geometric intuition to RKHS learning

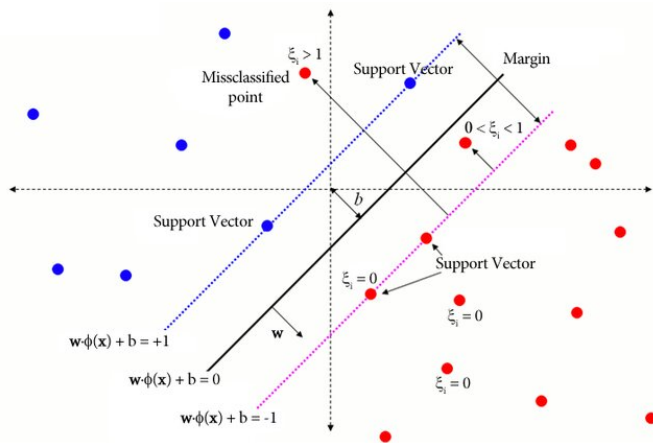


Figure: Points well-classified
lying outside or on the margin:
 $\xi_i = 0$.

Points lying inside the margin/tube, but still well-classified: $0 < \xi_i \leq 1$.

Points lying on the wrong side of the decision boundary (misclassified): $\xi_i > 1$.

From geometric intuition to RKHS learning

$$\begin{aligned} \min_{(w,b,\xi) \in H_0 \times \mathbb{R} \times \mathbb{R}_+^m} \quad & \langle w, w \rangle + C \sum_{i=1}^m \xi_i \\ \text{subject to:} \quad & y_i(\langle w, \Phi(x_i) \rangle + b) \geq 1 - \xi_i \quad i = 1, \dots, m \end{aligned}$$

- Notice that the constraint on ξ_i (together with its domain) is equivalent to

$$\xi_i \geq \max\{0, 1 - y_i(\langle w, \Phi(x_i) \rangle + b)\} =: \ell(\langle w, \Phi(x_i) \rangle + b, y_i) =: \ell(f_{(w,b)}(x_i), y_i).$$

with the *hinge-loss* $\ell(y, y') = \max\{0, 1 - yy'\}$ and the function $f_{(w,b)}(x) = \langle w, \Phi(x) \rangle + b$.

- Since at optimality, the constraint is binding, we can equivalently write the optimization as

$$\min_{(w,b) \in H_0 \times \mathbb{R}} \lambda \langle w, w \rangle + \frac{1}{m} \sum_{i=1}^m \ell(f_{(w,b)}(x_i), y_i)$$

with $\lambda = 1/(mC)$.

From geometric intuition to RKHS learning

$$\min_{(w,b) \in H_0 \times \mathbb{R}} \lambda \langle w, w \rangle + \frac{1}{m} \sum_{i=1}^m \ell(f_{(w,b)}(x_i), y_i)$$

- Recall $f_{(w,b)}(x_i) = \langle w, \Phi(x_i) \rangle_{H_0} + b$. We will use (and later find) that the space of functions

$$H = \{f : \mathbb{R}^d \rightarrow \mathbb{R} : \exists w \in H_0 \text{ with } f = \langle w, \Phi(\cdot) \rangle_{H_0}\}$$

will itself be a Hilbert space, with norm $\|f\|_H = \|w\|_{H_0}$.

- We can rewrite the optimization problem as

$$\min_{(f,b) \in H \times \mathbb{R}} \lambda \|f\|_H^2 + \frac{1}{m} \sum_{i=1}^m \ell(f(x_i) + b, y_i)$$

- For the sake of simplicity, we will leave out the constant b and arrive at the standard form for empirical kernel regression

$$\min_{f \in H} \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i) + \lambda \|f\|_H^2$$

From classification to regression

- For regression, the goal is to find a decision hyperplane $(w, b) \in H_0 \times \mathbb{R}$ that instead best approximates the relationship between lifted inputs $\Phi(x) \in H_0$ and **continuous output** $y \in \mathbb{R}$.
- Support Vector Regression tries to fit the data within a "tube" of a specified width ε around the hyperplane. Points that lie inside the tube are not penalized, while points outside incur a loss, determined by how far they fall outside the margin.
- We can rewrite the adjusted optimization problem (2) for regression as

$$\min_{(w, b, \xi_i) \in H_0 \times \mathbb{R} \times \mathbb{R}_+^m} \langle w, w \rangle + C \sum_{i=1}^m \xi_i$$

subject to: $|y_i - \langle w, \Phi(x_i) \rangle - b| \leq \varepsilon + \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, m$

where $\varepsilon \geq 0$ is fixed and defines the ε -tube within which no penalty is applied.

- Similarly to classification, we rewrite the optimization problem in $H = \{f : f = \langle w, \Phi(\cdot) \rangle_{H_0}\}$,

$$\min_{(f, b) \in H \times \mathbb{R}} \lambda \|f\|_H^2 + \frac{1}{m} \sum_{i=1}^m \ell_\varepsilon(f(x_i) + b, y_i)$$

where ℓ_ε now denotes the ε -insensitive loss function $\ell_\varepsilon(y, y') = \max\{0, |y - y'| - \varepsilon\}$.

- The square loss $\ell(y, y') = (y' - y)^2$ is also a popular choice for regression, see Corollary 5.8.

From classification to regression

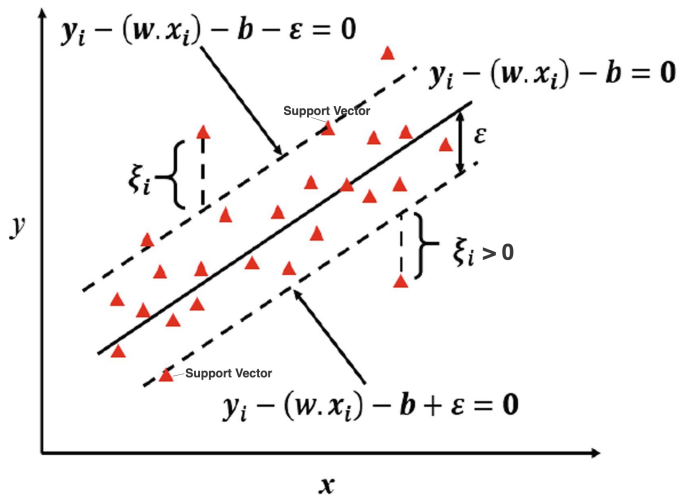


Figure: Support Vector Regression for $\Phi(x) = x$.

Again, the slack variables $\xi_i \geq 0$ allow observations to lie outside the ε -tube such as to prevent overfitting, but at the cost of a penalty in the objective function.

RKHS learning: Importance of the kernel

The previous discussion is quite abstract in the following ways:

- What is the feature map Φ ?
- How to choose the space H_0 ?
- What is the resulting space of functions H ?

Questions are made easier through the use of **kernel functions**.

Definition 5.1

A function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a kernel on $\mathcal{X}$ if there exists a Hilbert space H and a map $\Phi : \mathcal{X} \rightarrow H$ such that for all $x, x' \in \mathcal{X}$ we have $k(x, x') = \langle \Phi(x), \Phi(x') \rangle$.

Lemma 5.2

A function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a kernel if and only if it is symmetric and positive semidefinite, where the latter means that for all $n \in \mathbb{N}$, $\alpha_1, \dots, \alpha_n \in \mathbb{R}$ and $x_1, \dots, x_n \in \mathcal{X}$ we have

$$\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j k(x_j, x_i) \geq 0. \quad (3)$$

Examples of kernel functions

Example 5.3

The following are kernel functions on $\mathbb{R}^d$ (we use the notation $x, x' \in \mathbb{R}^d$):

- (i) *Linear kernel:* $k(x, x') = \langle x, x' \rangle$
- (ii) *Polynomial kernel:* for some $p \in \mathbb{N}$ and $c \in \mathbb{R}_+$, $k(x, x') = (\langle x, x' \rangle + c)^p$
- (iii) *Gaussian kernel:* for $\gamma > 0$, $k(x, x') = \exp(-\|x - x'\|_2^2 / \gamma^2)$
- (iv) *Compactly supported radial basis kernel:* for $p > d/2$, $k(x, x') = \max\{1 - \|x - x'\|_2, 0\}^p$

In general, kernels of the form $k(x, x') = \phi(\|x - x'\|_2)$ for $\phi : \mathbb{R}_+ \rightarrow \mathbb{R}$ are called *radial basis function kernels*. A scaling factor γ so that $k_\gamma(x, x') = \phi(\|x - x'\|_2 / \gamma)$ is called the *bandwidth*.

Proof.

(i)-(iii) are part of problem set 3.

(iv) are particular cases of compactly supported radial basis function kernels of minimal degree, cf. Wendland, 1998. □

Reproducing kernel Hilbert spaces

Let H be a (as always, real) Hilbert space consisting of functions $f : \mathcal{X} \rightarrow \mathbb{R}$.

Definition 5.4

- (i) We call H a reproducing kernel Hilbert space (RKHS), if $\delta_x : H \rightarrow \mathbb{R}$ given by $\delta_x(f) := f(x)$ is continuous for all $x \in \mathcal{X}$.
- (ii) A function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a reproducing kernel of H , if $k(\cdot, x) \in H$ and $f(x) = \langle f, k(\cdot, x) \rangle$ for all $x \in \mathcal{X}, f \in H$. If such a function k exists, we call $\Phi : \mathcal{X} \rightarrow H$ given by $\Phi(x) := k(\cdot, x)$ the canonical feature map (i.e. reproducing kernels are kernels).

Theorem 5.5

- (i) If H has a reproducing kernel k , then H is a RKHS. (proof in class)
- (ii) Every RKHS has a unique reproducing kernel k . (proof in class)
- (iii) For every kernel k , there exists a unique RKHS having k as its reproducing kernel.

(iii) is shown in Steinwart and Christmann, Th. 4.21: Let k be a kernel over $\mathcal{X}$ with feature map $\Phi : \mathcal{X} \rightarrow H_0$. Then $H := \{f : \mathcal{X} \rightarrow \mathbb{R} : \exists w \in H_0 \text{ with } f = \langle w, \Phi(\cdot) \rangle_{H_0}\}$ equipped with the norm $\|f\|_H = \|w\|_{H_0}$ is the only RKHS for which k is a reproducing kernel. The resulting RKHS can then be made explicit by Mercer's theorem for $\mathcal{X}$ compact!

Mercer's theorem

Let $\mathcal{X}$ be compact, $\mu \in \mathcal{P}(\mathcal{X})$ have full support, and assume $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a continuous kernel. We define the kernel integral operator $T_{k,\mu} : L^2(\mu) \rightarrow L^2(\mu)$ via

$$T_{k,\mu}f(x) = \int k(x,y)f(y) \mu(dy).$$

The spectral theorem (Theorem A.5.13 in Steinwart and Christmann, 2008) shows

$$T_{k,\mu}f = \sum_{i=1}^{\infty} \lambda_i \langle f, \tilde{\varphi}_i \rangle \tilde{\varphi}_i \quad \text{for all } f \in L^2(\mu)$$

for eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$ and orthonormal (in $L^2(\mu)$) eigenfunctions $(\tilde{\varphi}_i)_{i \in \mathbb{N}} \subset H$ of $T_{k,\mu}$.

Theorem 5.6 (Mercer's theorem; stated for the case $\lambda_i > 0$ for all $i \in \mathbb{N}$)

There are continuous representatives φ_i of $\tilde{\varphi}_i$ such that $k(x, x') = \sum_{i=1}^{\infty} \lambda_i \varphi_i(x) \varphi_i(x')$, $x, x' \in \mathcal{X}$. Further, there exists a RKHS $(H, \langle \cdot, \cdot \rangle_H)$ corresponding to k , which is given by

$$H = \left\{ \sum_{i=1}^{\infty} a_i \varphi_i : a_i \in \mathbb{R}, i \in \mathbb{N}, \text{ with } \sum_{i=1}^{\infty} \frac{a_i^2}{\lambda_i} < \infty \right\} \quad \text{and} \quad \langle f, g \rangle_H = \sum_{i=1}^{\infty} \frac{a_i b_i}{\lambda_i}$$

for $f, g \in H$, $f = \sum_{i=1}^{\infty} a_i \varphi_i$, $g = \sum_{i=1}^{\infty} b_i \varphi_i$.

Mercer's theorem

The proof of Mercer's theorem goes beyond what we can cover in this section, but we give references for completeness.

Proof.

For the pointwise representation of k , refer to Theorem 4.49 in Steinwart and Christmann, 2008. Regarding the construction and properties of the given H , see Theorem 4.51 in Steinwart and Christmann, 2008. □

Solving learning problems over RKHS

We now come to actually solving, for $(x_1, y_1), \dots, (x_m, y_m) \in \mathbb{R}^d \times \mathbb{R}$, the learning problem

$$\min_{f \in H} \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i) + \lambda \|f\|_H^2.$$

Theorem 5.7 (Representer theorem)

Let $\ell : \mathbb{R} \times \mathbb{R} \rightarrow [0, \infty)$ be continuous in its first argument and H be the RKHS of a measurable kernel k . Then, there exists a function $f_m^ \in H$ such that*

$$\frac{1}{m} \sum_{i=1}^m \ell(f_m^*(x_i), y_i) + \lambda \|f_m^*\|_H^2 = \inf_{f \in H} \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i) + \lambda \|f\|_H^2,$$

and for any such optimizer $f_m^ \in H$, there exist $\alpha_1, \dots, \alpha_m \in \mathbb{R}$ such that*

$$f_m^*(x) = \sum_{i=1}^m \alpha_i k(x_i, x) \quad \text{for all } x \in \mathcal{X}.$$

Moreover, if ℓ is convex in its first argument, the optimizer is unique.

Representer theorem: Recap of proof

- **Objective:** Solving the optimization problem

$$\min_{f \in H} \quad \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i) + \lambda \|f\|_H^2.$$

- **Representation:** The main part of the representer theorem is that we can without loss of generality restrict the optimization to functions $f \in H_{X'}$ with

$$H_{X'} = \left\{ \sum_{i=1}^m \alpha_i k(x_i, \cdot) : \alpha_1, \dots, \alpha_m \in \mathbb{R} \right\}.$$

- **Existence:** Note that $g = 0 \in H$ gives a finite objective value $V_0 < \infty$. We can hence reduce the optimization to $f \in H_{X'}$ with $\|f\|_H^2 \leq V_0/\lambda \rightsquigarrow$ bounded and closed subset of a finite-dim. subspace of H (hence compact w.r.t. $\|\cdot\|_H$) $\rightsquigarrow$ existence follows since ℓ is continuous.
- **Uniqueness:** If ℓ is convex we obtain uniqueness due to strict convexity of $f \mapsto \|f\|_H^2$.

Solution for squared loss function

Corollary 5.8 (to the representer theorem)

Let $\ell(y, y') = (y - y')^2$. Assume $x_1, \dots, x_m$ are distinct and k is positive definite. Then, the parameters $a = (\alpha_1, \dots, \alpha_m)$ of the optimizer $f_m^ = \sum_{i=1}^m \alpha_i k(x_i, \cdot)$ from Theorem 5.7 are given by*

$$a = (\lambda m I_m + K_m)^{-1} b,$$

where $I_m \in \mathbb{R}^{m \times m}$ is the identity matrix, $K_m \in \mathbb{R}^{m \times m}$ is given by $K_m[i, j] = k(x_i, x_j)$, and $b = (y_1, \dots, y_m)$.

END OF FIRST PART

So far...

- Geometric idea behind kernel methods (linear prediction function with possibly infinite-dimensional features)
- Definition of RKHS and importance of kernel
- With the representer theorem, we don't need to explicitly work with the latent space H at all since all necessary calculations can be done with the kernel $\rightsquigarrow$ so-called *kernel trick*

To conclude the part on SVM/RKHS, we take a brief look at the following questions:

- What are the state of the art algorithmic approaches with kernel methods?
- Can we approximate arbitrary continuous functions with RKHS functions?
- How data-efficient is kernel learning?
- What can other methods offer beyond kernel methods?

Numerical approaches

- For the **square loss** ($\ell(y, y') = |y' - y|^2$) we have seen the closed form solution.
Implementation: sklearn package in python (`sklearn.kernel_ridge.KernelRidge`)
Limitations: Memory requirements are quadratic in data set. No sparsity in solution.
Advantage: Computationally efficient thanks to Corollary 5.8.
- For the **ε -insensitive loss** ($\ell_\varepsilon(y, y') = \max\{0, |y' - y| - \varepsilon\}$, $\varepsilon \geq 0$ fixed), one can apply a variant of the SMO algorithm to general regression problems. (See Smola and Schölkopf, 2004)
Implementation: sklearn package (`sklearn.svm.SVR`)
Advantage: More robust to outliers and leads to sparse solution.
Limitations: More complex optimization problem since ℓ_ε not differentiable at $|y' - y| = \varepsilon$.
- For **binary classification with the hinge loss** ($\ell(y, y') = \max\{0, 1 - yy'\}$) the SMO (Sequential Minimal Optimization) algorithm is usually used. The SMO algorithm is a form of coordinate descent, where in each step two indices $(i_1, i_2) \in \{1, \dots, m\}$ are chosen and one *only* optimizes for $(\alpha_{i_1}, \alpha_{i_2})$. (See Platt, 1998) **More generally:** SMO belongs to a class of algorithms which are based on solving simpler sub-problems, referred to as *decomposition methods*, *chunking* or *subset selection*.
Implementation: sklearn package (`sklearn.svm.SVC`)
Advantage: For large datasets, many α_i may simply remain 0 $\Rightarrow$ sparse solution

SMO algorithm for binary classification

Let $\ell(y, y') = \max\{0, 1 - yy'\}$. Using geometric intuition & representer theorem ($f_\alpha = \sum_{i=1}^m \alpha_i k(x_i, \cdot)$, $C > 0$), the empirical RKHS problem (with constant) is equivalent to:

$$\begin{aligned} \min_{(\alpha, \xi, b) \in \mathbb{R}^m \times \mathbb{R}_+^m \times \mathbb{R}} \quad & \|f_\alpha\|_H^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to:} \quad & y_i(f_\alpha(x_i) + b) \geq 1 - \xi_i \quad i = 1, \dots, m \end{aligned}$$

The Lagrangian is given by

$$\min_{(\alpha, \xi, b) \in \mathbb{R}^m \times \mathbb{R}_+^m \times \mathbb{R}} \max_{(\eta, w) \in \mathbb{R}_+^{2m}} \|f_\alpha\|_H^2 + C \sum_{i=1}^m \xi_i - \sum_{i=1}^m \eta_i \xi_i - \sum_{i=1}^m w_i (y_i(f_\alpha(x_i) + b) - 1 + \xi_i).$$

FOC w.r.t. α , ξ and b yield the dual formulation (*quadratic program with box constraint*):

$$\max_{\substack{w \in [0, C]^m: \\ \sum_{i=1}^m w_i y_i = 0}} \sum_{i=1}^m w_i - \frac{1}{4} \sum_{i=1}^m \sum_{j=1}^m w_i w_j y_i y_j k(x_i, x_j)$$

SMO algorithm for binary classification

$$\max_{\substack{w \in [0, C]^m: \\ \sum_{i=1}^m w_i y_i = 0}} \sum_{i=1}^m w_i - \frac{1}{4} \sum_{i=1}^m \sum_{j=1}^m w_i w_j y_i y_j k(x_i, x_j) =: \max_{\substack{w \in [0, C]^m: \\ \sum_{i=1}^m w_i y_i = 0}} V(w)$$

Algorithm 5.9 (SMO algorithm from Platt, 1998)

- 1 Initialize $w_1^{(0)} = w_2^{(0)} = \dots = w_m^{(0)} = 0$.
- 2 Iterate $j = 1, 2, \dots$ until convergence criterion satisfied (e.g., FOC approximately satisfied):
 - (2.1) Choose indices $i_1, i_2 \in \{1, \dots, m\}, i_1 \neq i_2$ according to a given rule (usually a heuristic)
 - (2.2) Set $w_i^{(j)} := w_i^{(j-1)}$ for $i \neq i_1, i_2$
 - (2.3) Set
$$(w_{i_1}^{(j)}, w_{i_2}^{(j)}) := \arg \max_{\substack{(w_{i_1}, w_{i_2}) \in [0, C]^2: \\ (w_{i_1} - w_{i_1}^{(j-1)})y_{i_1} + (w_{i_2} - w_{i_2}^{(j-1)})y_{i_2} = 0}} V((w_i^{(j)})_{i \in \{1, \dots, m\} \setminus \{i_1, i_2\}}, w_{i_1}, w_{i_2})$$
 - (2.4) Update b according to FOC
- 3 Output optimal function $f_\alpha = \sum_{i=1}^m \frac{1}{2} w_i^{(j)} y_i k(x_i, \cdot) + b$ (where $\alpha_i = \frac{1}{2} w_i y_i$ follows from FOC)

Note: The algorithm is efficient because step (2.3) allows for an explicit solution.

Numerical approaches: Large scale methods for square loss

Analytic solution for the **square loss** is not feasible for large data sets. Resolutions:

- **Feature selection / random feature methods:** The basic idea is to select a subset $I \subset \{1, \dots, m\}$, $|I| < m$ and build an estimator of the form

$$\hat{f}(x) = \sum_{i \in I} \alpha_i k(x_i, x)$$

Methods differ in choosing the set I , for instance with the Nyström method (Williams and Seeger, 2000) or random feature methods (Rahimi and Recht, 2007)

Implementation: sklearn package (sklearn.kernel_approximation.Nystroem)

- **Preconditioning methods:** The analytic solution from Corollary 5.8 was given by $Aa = b$ for $A = (\lambda m I_m + K_m)$, $a = (\alpha_1, \dots, \alpha_m)$ and $b = (y_1, \dots, y_m)$. A popular preconditioning method called FALKON (Rudi, Carratino, and Rosasco, 2017) aims at finding suitable B such that

$$BB^\top = A^{-1} \quad \text{and} \quad B^\top A B y = B^\top b.$$

The solution y yields a solution a of $Aa = b$ via $a = By$. While constructing such B precisely is costly, one instead aims at finding B which satisfies $BB^\top \approx A^{-1}$ only approximately (again using random feature methods like Nyström approximation).

Implementation: Python implementation available from falconml.github.io/falcon/

Function approximation with RKHS

Let k be continuous and $\mathcal{X}$ be a compact subspace of $\mathbb{R}^d$.

Definition 5.10

We say that a RKHS H (or, equivalently, the kernel k) is universal, if H is dense in the set of all continuous functions with respect to $\|\cdot\|_\infty$, i.e., if for every continuous function $f : \mathcal{X} \rightarrow \mathbb{R}$ and $\varepsilon > 0$, there exists $h \in H$ such that

$$\|h - f\|_\infty \leq \varepsilon.$$

Example 5.11

The following are examples of universal kernels

- *Exponential kernel: $k(x, x') = \exp(\gamma \langle x, x' \rangle)$ for $\gamma > 0$*
- *Gaussian kernel: $k(x, x') = \exp(-\gamma \|x - x'\|_2^2)$ for $\gamma > 0$*
- *Binomial kernel: $k(x, x') = (1 - \langle x, x' \rangle)^{-\alpha}$ for $\alpha > 0$, $\mathcal{X} \subset \{x \in \mathbb{R}^d : \|x\|_2 < 1\}$*

For a proof, see Corollary 4.58 in the book by Steinwart and Christmann, 2008.

Takeaway message: Many RKHS spaces are quite large!

Function approximation with RKHS: Sample efficiency

k Gaussian kernel (other smooth universal kernels would work as well) and $\ell(y, y') = |y - y'|^2$.

Theorem 5.12 (Worst-case learning rate for kernel regression)

Let ρ be "sufficiently smooth" and the optimal regression function $\bar{f}$ be s -times continuously differentiable. Let $(X_i, Y_i)_{i=1}^\infty$ iid, $(X_1, Y_1) \sim \rho$, and $\hat{f}_m^*$ the optimal empirical kernel regression function with data $(X_i, Y_i)_{i=1}^m$ and $\lambda_m := m^{-d/(2s+d)}$. Then there exists $C > 0$ such that

$$\lim_{m \rightarrow \infty} \mathbb{P} \left(\|\hat{f}_m^* - \bar{f}\|_2 \geq Cm^{-\frac{s}{2s+d}} \right) = 0.$$

Theorem 5.13 (Minimax lower rates of convergence)

Let $\{\rho_\theta : \theta \in \Theta\}$ be a sufficiently rich family of smooth measures (containing sufficiently different $\bar{f}_\theta$ which are s -times continuously differentiable). Then, there exists $c > 0$ such that

$$\lim_{m \rightarrow \infty} \inf_{\hat{T}_m} \sup_{\theta \in \Theta} \mathbb{P} \left(\|\hat{T}_m - \bar{f}_\theta\|_2 \geq cm^{-\frac{s}{2s+d}} \right) = 1$$

where $\bar{f}_\theta$ is the optimal least square regression function for ρ_θ and the infimum is over all estimation methods $\hat{T}_m$ using iid data $(X_i^\theta, Y_i^\theta) \sim \rho_\theta$, $i = 1, \dots, m$.

Function approximation with RKHS: Sample efficiency

Kernel rate:

$$\lim_{m \rightarrow \infty} \mathbb{P} \left(\|\hat{f}_m^* - \bar{f}\|_2 \geq C m^{-\frac{s}{2s+d}} \right) = 0.$$

Minimax rate: (lower bound for *any* estimation method)

$$\lim_{m \rightarrow \infty} \inf_{\hat{T}_m} \sup_{\theta \in \Theta} \mathbb{P} \left(\|\hat{T}_m - \bar{f}_\theta\|_2 \geq c m^{-\frac{s}{2s+d}} \right) = 1$$

Proof: Both results are fairly classical by now, see e.g., (Steinwart, Hush, Scovel, et al., 2009) for the kernel bound, and (Stone, 1982) for the Minimax rate.

Discussion:

- For sufficiently "bad" distributions ρ , no learning algorithm can achieve a better learning rate than kernel regression!
- However, $m^{-\frac{s}{2s+d}}$ is a very bad rate in high dimensions (also: the constant C may be large)
- The "hope" is that real world data ρ exhibits lower dimensional structure and that the chosen learning algorithm is suitable to exploit this lower dimensional structure. → **In this regard, neural networks or tree-based methods are often better than kernel methods!**

Kernel methods compared to other methods

We have seen that...

- (i) Working with kernel functions is analytically and numerically tractable
- (ii) Kernel learning exhibits statistically optimal properties

What can different methods offer beyond that? What are problems of kernel methods?

- (a) For kernel methods, the number of parameters $(\alpha_1, \dots, \alpha_m)$ grows with the number of data points (disadvantage compared to neural network methods)
- (b) Kernel must be chosen from the start and strongly influences solution quality (partial resolution: multiple kernel learning, cf. Sonnenburg et al., 2006)
- (c) Even though learning rates using kernels are optimal in general (meaning: there are adversarial examples where nothing better is possible), in practically relevant situations **other learning methods can adapt to the structure of the data better than kernel methods**. For instance neural networks can be seen as kernel methods where the feature map is learned from data and thus better than any a-priori fixed kernel.

List of references I

- Platt, John (1998). “Sequential minimal optimization: A fast algorithm for training support vector machines”. In:
- Rahimi, Ali and Benjamin Recht (2007). “Random features for large-scale kernel machines”. In: *Advances in neural information processing systems* 20.
- Rudi, Alessandro, Luigi Carratino, and Lorenzo Rosasco (2017). “Falkon: An optimal large scale kernel method”. In: *Advances in neural information processing systems* 30.
- Smola, Alex J and Bernhard Schölkopf (2004). “A tutorial on support vector regression”. In: *Statistics and computing* 14, pp. 199–222.
- Sonnenburg, Sören et al. (2006). “Large scale multiple kernel learning”. In: *The Journal of Machine Learning Research* 7, pp. 1531–1565.
- Steinwart, Ingo and Andreas Christmann (2008). *Support vector machines*. Springer Science & Business Media.
- Steinwart, Ingo, Don R Hush, Clint Scovel, et al. (2009). “Optimal Rates for Regularized Least Squares Regression.”. In: *COLT*, pp. 79–93.
- Stone, Charles J (1982). “Optimal global rates of convergence for nonparametric regression”. In: *The annals of statistics*, pp. 1040–1053.

List of references II

- Wendland, Holger (1998). “Error estimates for interpolation by compactly supported radial basis functions of minimal degree”. In: *Journal of approximation theory* 93.2, pp. 258–272.
- Williams, Christopher and Matthias Seeger (2000). “Using the Nystrom method to speed up kernel machines”. In: *Advances in neural information processing systems* 13.